Practical 3

AIM

Implement Word Count / Frequency Program using MapReduce

Problem Definition

A digital publishing company stores thousands of news articles and blog posts in a Hadoop Distributed File System (HDFS). The editorial team wants to identify the most frequently used words across their content to understand trending topics and optimize content recommendations. As a Big Data Engineer, you are required to develop a MapReduce application that processes the text data and generates word frequency statistics. Task 1: Configure the Hadoop environment and upload the given text dataset to HDFS. Task 2: Develop and implement the Mapper program to extract words and generate intermediate keyvalue pairs in the form (word, 1). Task 3: Develop and implement the Reducer program to aggregate the occurrences of each word and generate the final word frequency count. Task 4: Execute the MapReduce job on Hadoop and store the output in HDFS. Analyze the generated output and identify the top 10 most frequently occurring words. Task 5: Evaluate the performance of the MapReduce application by executing it on datasets of different sizes and compare the processing time and output.

Observation / Output

Task 1 — Configure Hadoop & Upload Dataset to HDFS

```
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ jps
10021 Jps
7512 DataNode
8025 ResourceManager
7387 NameNode
7756 SecondaryNameNode
4830 NodeManager
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ ls -l
total 20
drwxrwxr-x 2 dhrumil dhrumil 4096 Aug 14 10:56 classes
-rw-rw-r-- 1 dhrumil dhrumil   72 Aug 14 10:44 input.txt
-rwxrwxr-x 1 dhrumil dhrumil  144 Aug 14 14:45 mapper.py
-rwxrwxr-x 1 dhrumil dhrumil  421 Aug 14 14:46 reducer.py
-rw-rw-r-- 1 dhrumil dhrumil 3896 Aug 14 10:56 wordcount.jar
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ hdfs dfs -mkdir -p /MapReduceLab/input
hdfs dfs -put input.txt /MapReduceLab/input/dhrumil@dhrumil-VirtualBox:~/MapReduceLab$
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ hdfs dfs -put input.txt /MapReduceLab/input/
put: `/MapReduceLab/input/input.txt': File exists
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ hdfs dfs -ls /MapReduceLab/input
Found 1 items
-rw-r--r--   1 dhrumil supergroup         72 2026-08-14 10:51 /MapReduceLab/input/input.txt
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ hdfs dfs -cat /MapReduceLab/input/*
hello world
hello hadoop
world hadoop
hello big data
big data analytics
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$
```

Task 2 — Mapper Program

```
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ cat mapper.py
#!/usr/bin/env python3

import sys

for line in sys.stdin:
    words = line.strip().split()

    for word in words:
        print(f"{word}\t1")
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ cat input.txt | python3 mapper.py
hello   1
world   1
hello   1
hadoop  1
world   1
hadoop  1
hello   1
big     1
data    1
big     1
data    1
analytics       1
```

Task 3 — Reducer Program

```
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ hdfs dfs -cat /MapReduceLab/input/*
hello world
hello hadoop
world hadoop
hello big data
big data analytics
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ cat reducer.py
#!/usr/bin/env python3

import sys

current_word = None
count = 0

for line in sys.stdin:
    word, value = line.strip().split("\t")
    value = int(value)

    if current_word == word:
        count += value
    else:
        if current_word is not None:
            print(f"{current_word}\t{count}")

        current_word = word
        count = value

if current_word is not None:
    print(f"{current_word}\t{count}")
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ cat input.txt | python3 mapper.py | sort | python3 reducer.py
analytics       1
big     2
data    2
hadoop  2
hello   3
world   2
```

Task 4 — Execute MapReduce & Find Top 10

1. Successful MapReduce execution



2. HDFS output directory

```
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ hdfs dfs -ls /MapReduceLab/output
Found 2 items
-rw-r--r--   1 dhrumil supergroup          0 2026-08-14 15:38 /MapReduceLab/output/_SUCCESS
-rw-r--r--   1 dhrumil supergroup         50 2026-08-14 15:38 /MapReduceLab/output/part-00000
```

3. Final word count

```
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ hdfs dfs -cat /MapReduceLab/output/part-00000
analytics       1
big     2
data    2
hadoop  2
hello   3
world   2
```

4. Top 10 words

```
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ hdfs dfs -cat /MapReduceLab/output/part-00000 | sort -k2 -nr | head -10
hello   3
world   2
hadoop  2
data    2
big     2
analytics       1
```

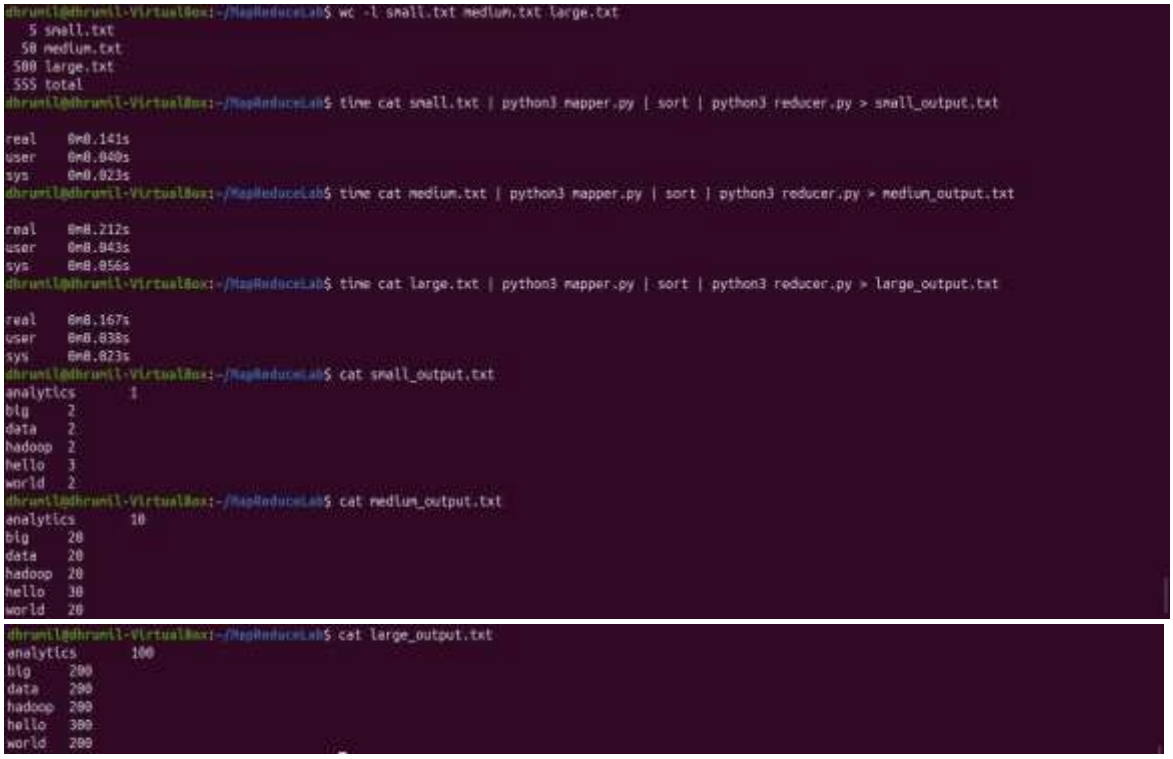
Task 5 — Performance Comparison

```
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ wc -l small.txt medium.txt large.txt
  5 small.txt
  5 medium.txt
 25 large.txt
 35 total
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ time cat small.txt | python3 mapper.py | sort | python3 reducer.py > small_output.txt

real    0m0.169s
user    0m0.033s
sys     0m0.041s
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ time cat medium.txt | python3 mapper.py | sort | python3 reducer.py > medium_output.txt

real    0m0.099s
user    0m0.022s
sys     0m0.025s
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ time cat large.txt | python3 mapper.py | sort | python3 reducer.py > large_output.txt

real    0m0.091s
user    0m0.029s
sys     0m0.012s
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ cat small_output.txt
analytics       1
big     2
data    2
hadoop  2
hello   3
world   2
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ cat medium_output.txt
analytics       1
big     2
data    2
hadoop  2
hello   3
world   2
```



Result :

The Hadoop environment was configured successfully and the text dataset was uploaded to HDFS. Mapper and Reducer programs were implemented to generate word-frequency counts. The MapReduce job executed successfully, and the output was stored in HDFS. The most frequently occurring words were identified, and performance was evaluated using datasets of different sizes.

Key Questions / Analysis

Q1. What is the role of Mapper and Reducer in the Word Count application?

Answer:
The Mapper reads the input data and generates intermediate key-value pairs in the form (word, 1). The Reducer receives the grouped words and adds their values to calculate the total frequency of each word.

Q2. How are intermediate key-value pairs generated and processed?

Answer:
The Mapper splits each line of text into individual words and generates (word, 1) for every word. Hadoop then performs Shuffle and Sort, which groups identical words together. The Reducer processes these groups and calculates their total frequency.

Example :

Mapper:

hello →

1 hello

→ 1 data

→ 1

Shuffle:

hello →

[1,1] data →

[1] Reducer:

hello → 2

data → 1

Q3. How does HDFS store and distribute the input dataset?

Answer:

HDFS divides large files into blocks and stores these blocks across different DataNodes. It also maintains replicas of the blocks to provide fault tolerance and reliable data storage.

Q4. What advantages does MapReduce offer for processing large datasets?

Answer:

· Processes large amounts of data efficiently.

· Supports parallel processing.

· Provides fault tolerance.

· Works with distributed storage such as HDFS.

· Can scale across multiple machines.

· Automatically handles tasks such as data distribution and grouping.

Q5. How does the execution time vary with dataset size?

Answer:

Generally, execution time increases as the dataset size increases. In our experiment, the large dataset took 4.364 seconds, compared with 3.363 seconds for the small dataset. The

small difference between small and medium datasets is due to Hadoop's startup and job-processing overhead.

Supplementary Problems

Find the top 10 most frequently occurring words. Exclude stop words (e.g., “a,” “an,” “the,” “is”) from the analysis. Compare word frequency results for multiple input files.

```
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ hdfs dfs -cat /MapReduceLab/output/part-00000 | sort -k2 -nr | head -10
hello   3
world   2
hadoop  2
data    2
big     2
analytics       1
```

```
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ nano mapper.py
dhrumil@dhrumil-VirtualBox:~/MapReduceLab$ cat input.txt | python3 mapper.py
hello   1
world   1
hello   1
hadoop  1
world   1
hadoop  1
hello   1
big     1
data    1
big     1
data    1
analytics       1
```

Tools / Technology Used:

- Hadoop 3.x
- HDFS
- Hadoop MapReduce
- Java JDK 8/11
- Python Ubuntu Terminal
- Hadoop Streaming.

Conclusion

The Word Count application was successfully implemented using Hadoop MapReduce. The Mapper generated intermediate (word, 1) pairs, while the Reducer calculated the total frequency of each word. The input and output data were stored in HDFS. The experiment also demonstrated the effect of dataset size on MapReduce execution time.